

PARIS CITE UNIVERSITY

RESEARCH PROJECT

Self-Supervised Test-Time Training for Image Classification

MAY 2026

Authors:

Rayane KHATIM

rayane.khatim@etu.u-paris.fr

22500285

Mehdi AGHAEI

mehdi.aghaei@etu.u-paris.fr

22500371

Supervisors:

Prof. Ayoub Karine : ayoub.karine@u-paris.fr

Prof. Camille Kurtz : camille.kurtz@u-paris.fr

Prof. Laurent Wendling : Laurent.Wendling@u-paris.fr

Draft prepared with current smoke-test and partial training results. Final numerical tables must be updated after the complete run finishes.

Contents

1	Introduction and Context	2
2	Scientific Background and Related Work	2
2.1	Supervised image classification and distribution shift	2
2.2	Self-supervised representation learning	3
2.3	Contrastive learning and NT-Xent loss	3
2.4	Linear evaluation	4
2.5	Test-Time Training and test-time adaptation	4
2.6	ActMAD and activation matching	4
3	Problem Formulation	5
4	Methodology	6
4.1	SimCLR pretraining on CIFAR-10	6
4.2	Feature extraction and projection head	6
4.3	Linear classifier on frozen features	6
4.4	Source activation statistics	7
4.5	ActMAD-style adaptation at test time	7
5	Implementation Details	7
5.1	Repository structure	7
5.2	Configuration files	8
5.3	Checkpoints and result files	9
5.4	CIFAR-10-C data path	9
5.5	Implementation issues to verify before the final run	10
6	Experimental Protocol	10
6.1	Source-domain pretraining	10
6.2	Linear evaluation	11
6.3	Test-time adaptation setup	11
6.4	All-corruption evaluation	11
7	Experimental Results	12
7.1	Current status of available results	12
7.2	Partial full-training loss log	12
7.3	Preliminary smoke TTT result	13
7.4	Final results table to update	13
8	Analysis and Discussion	14
9	Limitations	15
10	Reproducibility	16
11	Final Remarks and Future Work	17
A	Appendix: Final Update Checklist	17
B	Appendix: Result CSV Template	18

1 Introduction and Context

Image classification models usually work well when the test images are close to the images used during training. This assumption is weak in practice. A camera image can contain sensor noise, blur, compression artifacts, fog, snow, low contrast, or a strong change in brightness. These changes do not necessarily change the object class, but they change the input distribution seen by the model. A network trained only on clean images can then make unstable predictions, even when the image is still understandable for a human observer.

This project studies that problem with a small and controlled computer vision setting. CIFAR-10 is used as the clean source domain. The target domain is built from corrupted CIFAR-10 images, following the CIFAR-10-C evaluation idea. CIFAR-10-C is useful because it separates common corruptions into named corruption types and severity levels. It is not a question of adversarial attacks. The corruptions are more ordinary: noise, blur, weather effects, digital compression, contrast changes, and geometric distortions.

The project does not try to design a new backbone architecture. The backbone is ResNet18, and the representation is trained with SimCLR, a self-supervised contrastive method. The main question is narrower: after the model has been trained on the source data, can it adapt itself at test time using only an unlabeled corrupted batch? If this adaptation improves accuracy, then the model becomes less dependent on having perfectly clean test data. If it decreases accuracy, then the adaptation objective is not aligned well enough with classification.

The adaptation strategy implemented in the project is ActMAD-style activation matching. The model first records activation statistics on clean source data. At test time, it receives a corrupted batch and updates a small subset of parameters so that the target activations become closer to the source activations. The current implementation adapts normalization parameters and keeps the rest of the model frozen. The classification head is then applied after this short adaptation.

The current numerical state of the project is not final. A smoke test has finished and verifies that the SimCLR and TTT paths run end to end. A longer self-supervised run has also started, but the log provided so far stops at epoch 17 out of 100. For that reason, the results section is deliberately split into preliminary values and final-update placeholders. The smoke numbers are useful for validating the implementation. They are not enough for a final scientific conclusion.

2 Scientific Background and Related Work

2.1 Supervised image classification and distribution shift

In supervised image classification, the model learns a function

$$f_{\theta} : \mathcal{X} \rightarrow \{1, \dots, K\}$$

from images to class labels. For CIFAR-10, $K = 10$. The usual training objective assumes that the training and test examples are sampled from similar distributions. Under that condition, minimizing the training loss can lead to good test accuracy if the model generalizes correctly.

The problem changes when the test distribution is shifted. A corrupted image may still contain

the same semantic object, but the low-level statistics can be different. Noise modifies pixel values. Blur removes high-frequency details. Compression creates blocks and artifacts. Fog and brightness changes affect contrast and color distributions. Since convolutional neural networks learn a mixture of low-level and high-level features, these changes can propagate through the network and modify the internal activations.

CIFAR-10-C was introduced as part of the common corruptions benchmark proposed by Hendrycks and Dietterich [4]. The benchmark is useful because it avoids vague robustness claims. A method can be tested on fixed corruptions and severity levels. If the full benchmark is used, accuracy or error can be reported per corruption and then averaged. In this project, the official CIFAR-10-C format is treated as the correct final benchmark. Any local smoke or synthetic fallback must be reported separately.

2.2 Self-supervised representation learning

Self-supervised learning uses labels created from the data itself. The goal is not to predict the final class label directly, but to learn a representation that contains useful structure. After pretraining, a smaller supervised head can be trained on top of the frozen representation. This is often called linear evaluation when the head is a linear classifier.

SimCLR is a contrastive self-supervised method [1]. For each image, two different augmented views are generated. The model should map these two views close to each other in representation space. Views from different images should be separated. This gives a training signal without class labels. For this project, SimCLR is a good fit because CIFAR-10 can provide many augmented image pairs, and the learned backbone can later be reused for classification and test-time adaptation.

The implementation uses a ResNet18 backbone and a projection head. The backbone produces a feature vector. The projection head maps this feature vector into the contrastive space used for the SimCLR loss. The classifier is not trained during contrastive pretraining. It is trained later during linear evaluation on frozen features.

2.3 Contrastive learning and NT-Xent loss

Let x_i be an image and let t_1 and t_2 be two stochastic augmentations. SimCLR builds two views:

$$\tilde{x}_{i,1} = t_1(x_i), \quad \tilde{x}_{i,2} = t_2(x_i).$$

The encoder and projection head map them to normalized vectors $z_{i,1}$ and $z_{i,2}$. These two vectors are a positive pair. The other views in the same batch act as negative examples.

The project uses the NT-Xent loss. For a positive pair (i, j) , a simplified form is:

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k \neq i} \exp(\text{sim}(z_i, z_k)/\tau)},$$

where sim is a similarity score and τ is the temperature. In the code, the projected vectors are normalized and the similarity matrix is computed by a dot product. The diagonal is masked because each view should not be compared with itself. The target label for each view is the index of its

paired augmented view.

This loss is not a classification loss. It does not know the CIFAR-10 class names. It only asks the network to learn invariances induced by the augmentation pipeline. If the augmentations are well chosen, the representation captures useful image structure. If the augmentations are too weak or too strong, the representation can become less useful for classification.

2.4 Linear evaluation

Linear evaluation is a practical check of representation quality. After SimCLR pretraining, the backbone is frozen. A linear classifier is trained on top of the features using class labels. A high linear evaluation accuracy means that the frozen representation already separates classes well enough for a simple head. A low linear evaluation accuracy means that the representation is weak or not aligned with the classification task.

This matters for TTT. Test-time adaptation cannot fully repair a poor representation. If the backbone does not extract useful features before adaptation, then matching activation statistics may not be enough. The smoke run gave a linear evaluation accuracy of 17.19%, but this was after only one SSL epoch. This value should only be read as an execution check. The longer run was still training when the log was provided, so the final linear evaluation accuracy is still missing.

2.5 Test-Time Training and test-time adaptation

Test-Time Training adapts a model during inference using an auxiliary objective that does not need labels [2]. The general idea is simple. A model trained on a source domain receives target-domain inputs at test time. Before predicting labels, it performs a few gradient steps on a self-supervised or unsupervised loss. The hope is that this small adaptation moves the model toward the target distribution.

The difficulty is that the auxiliary loss is not the classification loss. It may improve the internal representation, but it may also move the model in a harmful direction. TTT++ studies this instability and shows that self-supervised test-time training can fail under some shifts [3]. This point is important for the discussion of our results. A positive average gain is not enough; the method must also be checked corruption by corruption.

Test-time adaptation is attractive because it does not require labeled target data. In many applications, labeled target data is not available when the distribution changes. The model only sees new unlabeled images. This matches the project setting: target batches from corrupted CIFAR-10 images are available, but their labels should not be used for adaptation.

2.6 ActMAD and activation matching

ActMAD proposes to adapt a model by matching activation distributions between source data and test-time data [5]. The main intuition is that a distribution shift changes internal activations. If the model can adjust itself so that target activations become closer to source activations, its predictions may become more stable.

The project implements a simplified ActMAD-style objective. During a source-statistics phase,

hooks record the activations of normalization layers on clean data. For each layer, the code computes a mean and a standard deviation. During adaptation, the target batch is passed through the model, target activation statistics are computed, and the loss penalizes the difference between target and source statistics. In simplified notation, for layers $l \in \mathcal{L}$:

$$\mathcal{L}_{\text{ActMAD}} = \frac{1}{|\mathcal{L}|} \sum_{l \in \mathcal{L}} (\|\mu_l^t - \mu_l^s\|_2^2 + \|\sigma_l^t - \sigma_l^s\|_2^2).$$

Here, μ_l^s and σ_l^s are source statistics. The target statistics μ_l^t and σ_l^t are computed on the current unlabeled target batch. This loss does not use the image labels.

3 Problem Formulation

The source domain is the clean CIFAR-10 distribution. It contains image-label pairs (x_s, y_s) sampled from $\mathcal{D}_s$. The target domain contains corrupted CIFAR-10 images sampled from $\mathcal{D}_t$. The labels still correspond to the same ten CIFAR-10 classes, but they are not used during adaptation.

The baseline prediction is obtained by applying the trained backbone and classifier directly to the corrupted batch:

$$\hat{y}_{\text{base}} = \arg \max f_{\theta}(x_t).$$

The adapted prediction first creates a temporary adapted model. A few gradient steps are applied on the unlabeled target batch using the activation matching loss. The prediction is then computed with the adapted parameters:

$$\hat{y}_{\text{ttt}} = \arg \max f_{\theta'}(x_t), \quad \theta' = \theta - \alpha \nabla_{\theta} \mathcal{L}_{\text{ActMAD}}(x_t).$$

In the current implementation, not all parameters are adapted. Most of the model is frozen, and only normalization layer parameters are updated.

The evaluation compares top-1 accuracy before and after adaptation. The improvement is reported in percentage points:

$$\Delta = 100 \times (\text{Acc}_{\text{TTT}} - \text{Acc}_{\text{base}}).$$

A positive value means the adaptation improved accuracy. A negative value means the adaptation damaged accuracy. This sign is important. Test-time adaptation should not be described as successful if it improves one corruption and fails on another.

The final report should distinguish three cases:

1. clean CIFAR-10 source training and linear evaluation;
2. official CIFAR-10-C evaluation, with official `.npy` corruption files;
3. smoke or fallback evaluation, useful for checking the code but not equivalent to the official benchmark.

The current draft only contains smoke results and a partial full-training log. Official full CIFAR-10-C results are still missing.

4 Methodology

4.1 SimCLR pretraining on CIFAR-10

The first stage trains a self-supervised encoder on CIFAR-10. The loader returns two augmented views of each image. The augmentations used in the repository include random resized cropping, horizontal flipping, color jitter, random grayscale conversion, conversion to tensor, and CIFAR-10 normalization. These transformations are defined in `src/datasets.py` through the `TwoViewsTransform` class.

The model is defined in `src/self_supervised.py`. It uses a torchvision ResNet18 with the final classification layer replaced by an identity mapping. The output of the backbone is then passed through a projection head. The projection head is a small multilayer perceptron with a linear layer, ReLU activation, and another linear layer. The output is normalized before the contrastive loss.

For each batch, the training loop computes two projected representations, one for each view. The NT-Xent loss is then applied. The optimizer used in the current code is Adam. A cosine annealing scheduler is also created. One implementation detail should be noted: the scheduler is defined with `T_max = epochs`, but in the inspected code it is stepped inside the mini-batch loop after each optimizer step. This may make the learning-rate schedule move faster than intended. It is not fatal for a smoke test, but it should be checked before final experiments.

4.2 Feature extraction and projection head

The backbone and projection head serve different roles. The projection head is useful for contrastive pretraining, but the classifier is trained on the raw backbone features. This is standard in SimCLR-style evaluation: the projection space helps the contrastive loss, while the representation before projection is used for downstream tasks.

In the code, `SimCLRModel.forward` returns the projected features for the SSL loss. The method `SimCLRModel.encode` returns the raw backbone features. Linear evaluation uses `encode`, not the projection head. This distinction is useful because it avoids forcing the final classifier to operate in the contrastive projection space.

4.3 Linear classifier on frozen features

After pretraining, the backbone parameters are frozen. The method `linear_eval` trains a linear layer with ten output classes. In the inspected implementation, this classifier is trained for 30 epochs with Adam and learning rate 10^{-3} . The classifier is saved as:

```
results/self_supervised/classifier.pt
```

The backbone checkpoint used for TTT is saved as:

```
results/self_supervised/simclr_backbone.pt
```

Linear evaluation is not the final project goal, but it gives an important diagnostic. If the linear classifier performs poorly, the TTT results should be interpreted carefully. A weak encoder can make adaptation unstable because the internal features are not strongly class-discriminative.

4.4 Source activation statistics

Before test-time adaptation, the project computes source activation statistics. The code registers forward hooks on BatchNorm and LayerNorm modules. For each selected layer, it records activations on clean source validation data and computes mean and standard deviation across the appropriate dimensions.

The result is saved as:

```
results/self_supervised/source_stats.pt
```

This file is required by the TTT stage. It represents the clean source-domain activation reference. During adaptation, the corrupted target batch is pushed toward these reference statistics.

4.5 ActMAD-style adaptation at test time

The TTT stage is implemented in `src/test_time_training.py`. The adapter stores the base model and source statistics. For each target batch, the adapter creates a deep copy of the base model. This is important because test-time updates should not permanently modify the original model across unrelated batches. Without a fresh copy, adaptation errors could accumulate from batch to batch.

The copied model is put in training mode. All parameters are frozen first. Then, only parameters inside BatchNorm, LayerNorm, and BatchNorm1d modules are made trainable. The adaptation optimizer is Adam. The current public TTT configuration uses:

```
adaptation:
  method: actmad
  steps_per_batch: 10
  lr: 0.0001
  source_stats_path: results/self_supervised/source_stats.pt
```

At each adaptation step, the model processes the unlabeled target batch, the activation matching loss is computed, and the normalization parameters are updated. After the adaptation steps, the copied model is used for prediction.

This design is cautious. It does not fine-tune the full network at test time. Updating all weights on a small unlabeled batch would be risky and expensive. Updating only normalization parameters is more limited, but it is less likely to destroy the learned representation.

5 Implementation Details

5.1 Repository structure

The project repository is organized around one entry point and several small modules. The main command-line file is `run.py`. It accepts two main tasks:

- `self_supervised`, also available as `ssl`;
- `test_time_training`, also available as `ttt`.

The README describes the repository as a Master 1 TER project focused on Test-Time Training and Self-Supervised Learning, with authors Rayane KHATIM and Mehdi AGHAEI. The main folders are `configs/`, `src/`, `scripts/`, `data/`, `papers/`, `docs/`, `notebooks/`, `presentation/`, and `report/`.

The implementation files used by the report are:

- `run.py`: argument parsing, task selection, checkpoint loading, and high-level execution;
- `src/self_supervised.py`: SimCLR model, projection head, NT-Xent loss, SSL trainer, linear evaluation, source-statistics call;
- `src/test_time_training.py`: activation hooks, source statistics, ActMAD loss, TTT adapter;
- `src/datasets.py`: CIFAR-10 loaders, SimCLR two-view transform, CIFAR-10-C loader;
- `src/metrics.py`: top-1 accuracy and baseline/TTT evaluation;
- `scripts/eval_corruption.py`: evaluation loop over the 15 CIFAR-10-C corruption names.

5.2 Configuration files

The public self-supervised configuration is `configs/self_supervised.yaml`. It uses CIFAR-10 at `data/raw`, enables download, and sets the training method to SimCLR. The current values in that file are:

Table 1: Self-supervised configuration found in `configs/self_supervised.yaml`.

Parameter	Value
Experiment name	<code>ssl_pretrain_baseline</code>
Seed	42
Dataset	<code>cifar10</code>
Dataset root	<code>data/raw</code>
Download	<code>true</code>
Backbone	<code>resnet18</code>
Embedding dimension	128
Method	<code>simclr</code>
Epochs	100
Batch size	256
Learning rate	0.03
Source statistics	<code>enabled</code>
Source stats path	<code>results/self_supervised/source_stats.pt</code>
Backbone path	<code>results/self_supervised/simclr_backbone.pt</code>

The public TTT configuration is `configs/test_time_training.yaml`. It uses CIFAR-10-C, corruption `shot_noise`, severity 5, batch size 16, ResNet18, ten classes, and the saved SimCLR backbone. The adaptation method is `actmad`, with ten steps per batch and learning rate 10^{-4} .

Table 2: TTT configuration found in `configs/test_time_training.yaml`.

Parameter	Value
Experiment name	<code>ttt_baseline_rotation</code>
Dataset	<code>cifar10c</code>
Dataset root	<code>data/raw/cifar10c</code>
Corruption	<code>shot_noise</code>
Severity	5
Batch size	16
Rotation shift	disabled
Backbone	<code>resnet18</code>
Number of classes	10
Checkpoint	<code>results/self_supervised/simclr_backbone.pt</code>
Adaptation method	<code>actmad</code>
Steps per batch	10
Adaptation learning rate	0.0001
Source stats path	<code>results/self_supervised/source_stats.pt</code>
Output directory	<code>results/test_time_training</code>

The smoke configuration files used in the latest terminal log were `configs/smoke_self_supervised.yaml` and `configs/smoke_test_time_training.yaml`. Their contents were not available in the inspected public repository snapshot. The report therefore uses only the exact metrics printed in the terminal log for the smoke test.

5.3 Checkpoints and result files

The report should track the following output paths:

```
results/self_supervised/best_backbone.pt
results/self_supervised/last_backbone.pt
results/self_supervised/simclr_backbone.pt
results/self_supervised/classifier.pt
results/self_supervised/source_stats.pt
results/self_supervised/training_log.csv
results/test_time_training/results.csv
```

The current public `run.py` saves `classifier.pt`, `simclr_backbone.pt`, and `source_stats.pt`. The files `best_backbone.pt`, `last_backbone.pt`, and `training_log.csv` are useful final-report targets, but their creation must be checked against the local version of the code. They are listed here because they are part of the project reporting plan.

5.4 CIFAR-10-C data path

The dataset loader expects official CIFAR-10-C files under:

```
data/raw/cifar10c/CIFAR-10-C/
```

For a corruption such as `shot_noise`, the expected file is:

```
data/raw/cifar10c/CIFAR-10-C/shot_noise.npy
```

The labels are expected in:

```
data/raw/cifar10c/CIFAR-10-C/labels.npy
```

Each corruption file contains 50,000 images, with 10,000 images per severity level. The loader selects the slice corresponding to the requested severity. For severity 5, it selects the last 10,000 images.

In the inspected public code, the CIFAR-10-C loader directly calls `np.load`. No fallback generator is visible in that file. If a local version of the project contains synthetic fallback corruptions for smoke tests, those results must be separated from official CIFAR-10-C results. A fallback is acceptable for checking that the pipeline runs, but it should not be presented as an official CIFAR-10-C benchmark.

5.5 Implementation issues to verify before the final run

The main pipeline is clear, but a few code details should be checked before submitting final results.

First, the learning-rate scheduler in `SelfSupervisedTrainer.pretrain` is stepped inside the mini-batch loop. Since the scheduler is initialized with `T_max=config.epochs`, it would be more coherent to step it once per epoch, or to set `T_max` to the total number of optimizer steps. This does not prevent the code from running, but it changes the actual learning-rate schedule.

Second, the helper `mean_corruption_error` in `src/metrics.py` appears to import `build_cifar10_loader`, while the dataset module provides `build_cifar10c_loader`. The main path used by `run.py` and `scripts/eval_corruption.py` uses the correct CIFAR-10-C loader, but this helper should still be corrected if mean corruption error is reported.

Third, the public `scripts/eval_corruption.py` prints results for all 15 corruptions, but the inspected code does not write `results/test_time_training/results.csv`. If the local version writes a CSV, it should be included in the final report. If not, CSV writing should be added so that the final tables are reproducible.

6 Experimental Protocol

6.1 Source-domain pretraining

The full source-domain pretraining command is:

```
python run.py --task self_supervised --config configs/self_supervised.yaml
```

The alias is:

```
python run.py --task ssl --config configs/self_supervised.yaml
```

The provided full-run log confirms that this command started on an Apple MPS device. The run uses the public self-supervised configuration with 100 epochs, batch size 256, learning rate 0.03,

SimCLR, CIFAR-10, and ResNet18. At the time of the provided log, training had reached epoch 17 out of 100. Final pretraining loss and final linear evaluation accuracy are not yet available.

6.2 Linear evaluation

After the SimCLR pretraining loop, `run.py` calls `linear_eval`. The current implementation freezes the backbone and trains a linear classifier for 30 epochs. The classifier is optimized with Adam at learning rate 10^{-3} . It is then evaluated on the CIFAR-10 validation/test loader returned by `build_cifar10_loaders`.

The smoke run produced a linear evaluation accuracy of 17.19% after one SSL epoch. This value is expected to be low because one epoch is not enough for a strong contrastive representation. The final full-run linear evaluation accuracy must replace this value once the 100-epoch run finishes.

6.3 Test-time adaptation setup

The TTT command is:

```
python run.py --task test_time_training --config configs/test_time_training.yaml
```

The alias is:

```
python run.py --task ttt --config configs/test_time_training.yaml
```

The public TTT configuration evaluates `shot_noise` at severity 5 with batch size 16. It loads the SimCLR backbone from `results/self_supervised/simclr_backbone.pt`, the classifier from `results/self_supervised/classifier.pt`, and the source activation statistics from `results/self_supervised/activation_stats.pt`. Adaptation uses ActMAD, 10 steps per batch, and learning rate 10^{-4} .

The smoke TTT command used `configs/smoke_test_time_training.yml`. Since the file content was not provided, the exact corruption, severity, batch size, and adaptation hyperparameters of the smoke run are not reported beyond what appears in the terminal output.

6.4 All-corruption evaluation

The repository includes a script for evaluating all CIFAR-10-C corruptions:

```
python scripts/eval_corruption.py
```

The script lists 15 corruptions:

```
gaussian_noise, shot_noise, impulse_noise, defocus_blur,
glass_blur, motion_blur, zoom_blur, snow, frost, fog,
brightness, contrast, elastic_transform, pixelate,
jpeg_compression
```

The severity is fixed to 5 in the inspected script. Each corruption should be evaluated with and without TTT. The final report should contain the resulting table once the full evaluation is available. At the moment, those full results are *Not provided in the available results*.

7 Experimental Results

7.1 Current status of available results

The current results are preliminary. The smoke test validates the execution path. The full SSL run is still incomplete. No final official CIFAR-10-C table was provided. The final report should therefore be updated when the complete logs, CSV files, and final metrics are available.

Table 3: Available numerical results at the current draft stage.

Item	Available value
Smoke SSL epoch count	1 epoch
Smoke SSL loss	4.0484
Smoke linear evaluation accuracy	17.19%
Smoke baseline accuracy without TTT	10.94%
Smoke ActMAD TTT accuracy	12.50%
Smoke improvement	+1.56 pp
Full SSL run status	Epoch 17/100 reached in the provided log
Final SSL loss	<i>Not provided in the available results.</i>
Final linear evaluation accuracy	<i>Not provided in the available results.</i>
Official CIFAR-10-C final table	<i>Not provided in the available results.</i>

The smoke result should be read carefully. A baseline close to 10% is close to random guessing for CIFAR-10. The TTT gain of +1.56 pp confirms that the adaptation path can change predictions, but it does not prove that the method is robust. A meaningful conclusion needs the completed backbone training and the full corruption evaluation.

7.2 Partial full-training loss log

The longer run used `configs/self_supervised.yaml` and printed the following loss values before the log stopped. The decreasing loss is a good sign, but the run is incomplete.

Table 4: Partial SimCLR training loss from the full run. The run was not finished when the log was provided.

Epoch	Loss	Epoch	Loss	Epoch	Loss
1	5.9363	7	5.2650	13	5.0246
2	5.7439	8	5.2073	14	5.0052
3	5.5509	9	5.1341	15	4.9896
4	5.4538	10	5.1049	16	4.9731
5	5.4032	11	5.0819	17	4.9579
6	5.3643	12	5.0464	—	—

The loss goes from 5.9363 at epoch 1 to 4.9579 at epoch 17. The improvement is clear, but

there is not yet enough evidence to know whether the representation has converged. The loss is still decreasing. The final checkpoint and final linear evaluation must be checked after epoch 100 or after early stopping if the local training version includes it.

Figure placeholder: final SimCLR training loss curve. Insert the plot from `results/self_supervised/training_log.csv` when the full run finishes. The current partial log is not enough for the final figure.

7.3 Preliminary smoke TTT result

The smoke TTT result is shown below. The exact smoke configuration file was not available, so the corruption and severity are not filled in. These values should not be mixed with the final official CIFAR-10-C evaluation.

Table 5: Preliminary smoke TTT result. This table is only an execution check.

Evaluation	Corruption / severity	Baseline	ActMAD TTT	Improvement
Smoke TTT run	<i>Not provided in the available results.</i>	10.94%	12.50%	+1.56 pp

This result is useful because it verifies the full path: checkpoint loading, classifier loading, source statistics loading, baseline prediction, adaptation, and adapted prediction. It is not enough for a claim about robustness. The baseline is very low, probably because the smoke backbone was trained for only one epoch. The correct next step is to repeat TTT after the full SSL training finishes.

7.4 Final results table to update

The final report should include one table with the official corruption results. This table is prepared below, but the values are intentionally left blank because they were not provided.

Table 6: Final official CIFAR-10-C results to update after full evaluation.

Corruption	Severity	Baseline accuracy	ActMAD TTT accu- racy	Improvement
gaussian_noise	5	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>
shot_noise	5	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>
impulse_noise	5	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>
defocus_blur	5	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>

Corruption	Severity	Baseline accuracy	ActMAD TTT accuracy	Improvement
glass_blur	5	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>
motion_blur	5	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>
zoom_blur	5	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>
snow	5	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>
frost	5	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>
fog	5	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>
brightness	5	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>
contrast	5	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>
elastic_transform	5	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>
pixelate	5	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>
jpeg_compression	5	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>
Mean	5	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>	<i>Not provided in the available results.</i>

8 Analysis and Discussion

The current smoke result gives only a weak signal. The adaptation improves accuracy from 10.94% to 12.50%, which is a gain of 1.56 percentage points. This shows that the ActMAD-style update is not doing nothing. The model copy is adapted, the activation loss is backpropagated, and the predictions change. However, the result is too close to random guessing to support a strong conclusion. A one-epoch SimCLR backbone is not a serious representation for CIFAR-10 classification.

The partial full-training loss is more informative. It decreases consistently over the first 17

epochs. This suggests that the SimCLR objective is being optimized. Still, contrastive loss alone does not directly tell us classification quality. The important number after pretraining is the linear evaluation accuracy. That number is missing for the current full run.

Once the final full run is finished, the first interpretation question will be representation quality. If the linear evaluation accuracy remains low, the final TTT results should be read as a test of the code rather than a strong test of the method. If the linear evaluation accuracy becomes reasonable, then the corruption results become more meaningful. TTT depends on the backbone. Activation matching can only help if the internal representation contains useful class information.

The second interpretation question is corruption dependence. TTT should be checked per corruption. Noise corruptions often disturb activation statistics strongly, so activation matching may help if the source statistics are informative. Brightness or fog changes may be different. They can shift image appearance without necessarily producing a simple correction in normalization statistics. In those cases, forcing activations toward the source distribution can damage information that the classifier still needs.

The third question is adaptation safety. Test-time adaptation is not automatically safe because the model adapts without labels. If the target batch is small, unbalanced, or too severely corrupted, the activation statistics can be misleading. The optimizer may move normalization parameters toward a lower ActMAD loss but worse classification accuracy. This is why the report should keep negative improvements visible. Removing bad corruptions from the table would give a false impression of the method.

The fourth question is batch size. The public TTT configuration uses batch size 16. This is practical on a local machine, but activation statistics from small batches can be noisy. Larger batches may give more stable estimates. Smaller batches may adapt faster but more randomly. This is a good hyperparameter to test if time allows.

The fifth question is the number of adaptation steps. The public configuration uses ten steps per batch. More steps may improve alignment, but they also increase the risk of over-adaptation. Fewer steps may be safer but weaker. A final version of the project would compare at least 1, 3, 5, and 10 steps on the same checkpoint and corruptions.

The current implementation is honest and understandable. It has the right components for the TER objective: SimCLR pretraining, frozen linear evaluation, source activation statistics, baseline evaluation, and test-time activation matching. The main weakness is not the concept. The main weakness is that final results are not yet available and some engineering details still need to be cleaned before the report can make strong claims.

9 Limitations

The first limitation is the current state of the results. The smoke test is not a final experiment. It used only one SSL epoch and produced a linear evaluation accuracy of 17.19%. That number is too low for a meaningful robustness study. The full 100-epoch run had not finished when the log was provided.

The second limitation is the training budget. SimCLR usually benefits from long training, large batches, and carefully tuned augmentations. The project is a Master 1 TER implementation

on local hardware, not a large-scale pretraining study. This should be stated directly. A weaker backbone can reduce the apparent benefit of TTT.

The third limitation is the learning-rate schedule implementation. The inspected code creates a cosine scheduler with `T_max` equal to the number of epochs, but calls the scheduler step inside the batch loop. This should be corrected or justified before final experiments, because it changes the effective schedule.

The fourth limitation is hyperparameter search. The report currently has one main TTT setting: ActMAD with ten adaptation steps and learning rate 10^{-4} . That is not enough to claim that the method has been optimized. The final report can still be strong if it is honest: it tests a reasonable implementation and discusses where it works or fails, but it does not claim exhaustive tuning.

The fifth limitation is the official CIFAR-10-C benchmark. The public loader expects the official `.npy` files. If final experiments are run on official CIFAR-10-C, the report can describe them as official. If they are run on local synthetic corruptions or a fallback, the report must say so clearly. A fallback is useful for testing code, but it is not equivalent to CIFAR-10-C.

The sixth limitation is adaptation instability. ActMAD aligns activation statistics, not labels. A lower activation matching loss does not guarantee better classification. This is not a bug; it is a known difficulty of test-time adaptation. Negative corruptions must remain in the results table.

10 Reproducibility

The main commands are short. From the repository root:

```
python run.py --task self_supervised --config configs/self_supervised.yaml
python run.py --task ssl --config configs/self_supervised.yaml
python run.py --task test_time_training --config configs/test_time_training.yaml
python run.py --task ttt --config configs/test_time_training.yaml
python scripts/eval_corruption.py
```

The expected order is:

1. run self-supervised training;
2. train and save the linear classifier;
3. save the SimCLR backbone;
4. compute and save source activation statistics;
5. run TTT on one corruption or all corruptions;
6. save the final CSV and update the report tables.

A minimal final-results checklist is:

- `results/self_supervised/simclr_backbone.pt` exists;
- `results/self_supervised/classifier.pt` exists;
- `results/self_supervised/source_stats.pt` exists;

- final linear evaluation accuracy is printed or stored;
- `results/test_time_training/results.csv` contains baseline and TTT accuracy per corruption;
- the report table uses the same values as the CSV;
- official CIFAR-10-C and fallback/smoke results are not mixed.

The terminal warnings observed during the provided run are not central to the scientific method. The MPS warning says that `pin_memory=True` is not supported on MPS, so pinned memory is not used. This can be cleaned by setting `pin_memory=False` when the device is MPS. The NumPy deprecation warning comes from dataset loading and does not change the reported accuracy values, but using a stable dependency combination is safer for the final run.

11 Final Remarks and Future Work

The project implements a complete pipeline for self-supervised test-time adaptation on image classification. SimCLR trains a ResNet18 backbone on CIFAR-10. A linear classifier evaluates the learned representation. Source activation statistics are computed on clean data. At test time, ActMAD-style adaptation updates normalization parameters on unlabeled corrupted batches before classification.

The current smoke test proves that the pipeline runs. It gives 17.19% linear evaluation accuracy after one SSL epoch and improves the smoke TTT accuracy from 10.94% to 12.50%. These numbers are not final. The full run is still in progress, with the provided log reaching epoch 17 out of 100 and showing a decreasing SimCLR loss.

The final version should add the completed linear evaluation accuracy, the official CIFAR-10-C table, mean baseline accuracy, mean TTT accuracy, and per-corruption improvements. The strongest future work is practical: run all 15 corruptions, several severity levels, and several adaptation-step settings. It would also be useful to compare ActMAD with another test-time adaptation method and to test safer stopping criteria that avoid harmful adaptation.

A Appendix: Final Update Checklist

Before submission, replace the following entries:

1. final SimCLR loss curve from `results/self_supervised/training_log.csv`;
2. final linear evaluation accuracy after the completed run;
3. final TTT result table from `results/test_time_training/results.csv`;
4. mean baseline accuracy, mean TTT accuracy, and mean improvement;
5. exact corruption source: official CIFAR-10-C or fallback/synthetic smoke data;
6. final hyperparameters if local configs differ from the public repository snapshot;
7. any code fixes applied before final experiments, especially the scheduler step and CSV saving.

B Appendix: Result CSV Template

The final CSV should contain at least these columns:

```
corruption,severity,batch_size,steps_per_batch,lr,
baseline_accuracy,ttt_accuracy,improvement_pp,
evaluation_source,checkpoint,notes
```

A clean final row should look like this format, with real values:

```
shot_noise,5,16,10,0.0001,0.1094,0.1250,1.56,
smoke_or_official,results/self_supervised/simclr_backbone.pt,
preliminary smoke result
```

References

- [1] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. A Simple Framework for Contrastive Learning of Visual Representations. In *Proceedings of the 37th International Conference on Machine Learning*, 2020.
- [2] Y. Sun, X. Wang, Z. Liu, J. Miller, A. A. Efros, and M. Hardt. Test-Time Training with Self-Supervision for Generalization under Distribution Shifts. In *Proceedings of the 37th International Conference on Machine Learning*, 2020.
- [3] Y. Liu, P. Kothari, B. G. van Delft, B. Bellot-Gurlet, T. Mordan, and A. Alahi. TTT++: When Does Self-Supervised Test-Time Training Fail or Thrive? In *Advances in Neural Information Processing Systems*, 2021.
- [4] D. Hendrycks and T. Dietterich. Benchmarking Neural Network Robustness to Common Corruptions and Perturbations. In *International Conference on Learning Representations*, 2019.
- [5] M. J. Mirza, P. Jané Soneira, W. Lin, M. Kozinski, H. Possegger, and H. Bischof. Act-MAD: Activation Matching to Align Distributions for Test-Time-Training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023.
- [6] A. Krizhevsky. Learning Multiple Layers of Features from Tiny Images. Technical report, University of Toronto, 2009.
- [7] K. He, X. Zhang, S. Ren, and J. Sun. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.